An integer square-root instruction for the CVE2 RISC-V core

Ahmed Ez-Zouine, M.Sc. 0009-0006-3939-0642 ahmedez-zouine.github.io/cve2-custom-isa-extension

Core CVE2, RV32IMC, 2-stage **FPGA** Artix-7 `xc7a35tcsg324-1`, 20 ns **Tools** Verilator 5.054, Vivado 2024.2

Summary. This report documents the design, RTL verification, and Artix-7 FPGA evaluation of a custom unsigned integer square-root instruction (`isqrt rd, rs1`) for the OpenHW Group CVE2 (`CV32E20`) RISC-V processor core. The instruction is implemented as a purely combinational, 16-stage restoring radix-4 datapath inside the existing ALU, requiring no new pipeline registers, no multi-cycle stall logic, and no modifications to the external core interface or simulation testbench. All supplied tests pass alongside two new verification suites that execute over 200,000 directed, boundary, random, and illegal-instruction trap checks. On the supplied 256-sample workload, replacing the software routine with `isqrt` reduces the average cost per root from 247 to 23 cycles (a 10.7× speed-up). Synthesising and routing the baseline and extended cores on a Xilinx Artix-7 (`xc7a35tcsg324-1`) shows the exact hardware cost of single-cycle unrolling: core LUT usage rises by 1,218 LUTs (+40.8%), only 4 flip-flops are added, and the 16 chained compare-and-subtract stages create a 49.387 ns critical path across 83 logic levels. We analyse this area–timing trade-off in detail and specify a 16-cycle iterative alternative that reuses a single step per clock cycle.

Keywords: RISC-V, CVE2, CV32E20, custom instruction extension, integer square root, RTL verification, FPGA synthesis.

1 Instruction specification and core architecture

For any unsigned 32-bit input $x \in [0, 2^{32} - 1]$ held in source register `rs1`, the custom instruction `isqrt rd, rs1` computes the exact floor of the square root and writes it to destination register `rd`:

$$\text{rd} \leftarrow r = \lfloor \sqrt{x} \rfloor, \qquad 0 \le r \le 65{,}535, \qquad r^2 \le x < (r+1)^2. \tag{1}$$

Because $\lfloor \sqrt{2^{32} - 1} \rfloor = 65{,}535 =$ `0x0000FFFF`, the result always fits in the lower 16 bits of `rd`, while bits `31:16` are zero-extended. The instruction is mapped to the RISC-V `custom-0` major opcode (`7'b0001011`), which the unprivileged ISA specification reserves specifically for non-standard user extensions [1].

Table 1. R-type encoding of `isqrt rd, rs1` in the `custom-0` opcode space. Any other bit pattern under `0001011` (`funct7 != 0`, `rs2 != 0`, or `funct3 != 0`) raises a synchronous illegal-instruction exception.

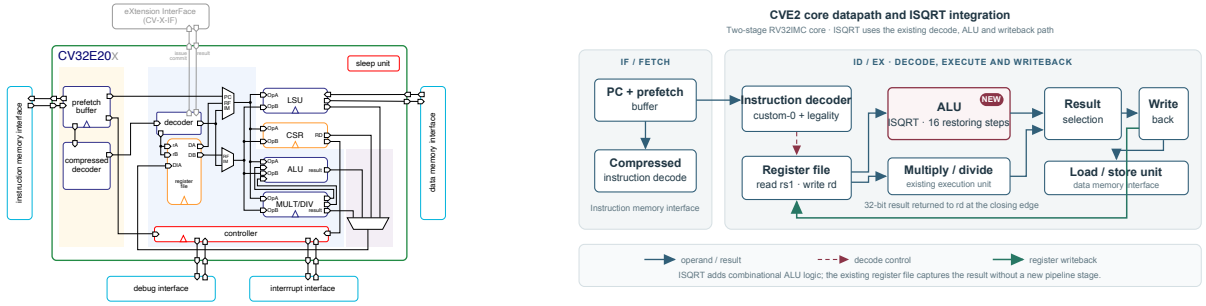
31:25 funct7	24:20 rs2	19:15	14:12 funct3	11:7	6:0 opcode
0000000	00000	rs1	000	rd	0001011

Because `isqrt` is a unary operation, the second source field `rs2` (`instr[24:20]`) is fixed to `5'b00000`, alongside `funct7 = 7'b0000000` and `funct3 = 3'b000`. Enforcing all three subfields in the decoder leaves the remaining `custom-0` encoding space clean for future instructions and guarantees that malformed binary words trap predictably.

Integration into the CVE2 two-stage pipeline

Figure 1 compares the full CVE2 (`CV32E20`) microarchitecture [2] with the specific datapath slice modified for `isqrt`. CVE2 is a compact, in-order, two-stage RISC-V core comprising an Instruction Fetch (IF) stage and a combined Instruction Decode / Execute (ID/EX) stage:

- **IF stage:** An instruction memory interface feeds a prefetch buffer and a compressed instruction decoder (`cve2_compressed_decoder.sv`), which expands 16-bit RV32C parcels into standard 32-bit instructions before registering them into the IF/ID pipeline stage.
- **ID/EX stage:** The instruction decoder (`cve2_decoder.sv`) drives the two-read, one-write register file (`cve2_register_file_ff.sv`), the main state controller (`cve2_controller.sv`), and the execution units: the ALU (`cve2_alu.sv`), the multiplier/divider block (`cve2_multdiv_fast.sv`), the Load-Store Unit (LSU), and the Control and Status Registers (CSRs).



(a) Full CVE2 (`CV32E20`) core block diagram [2]. **(b)** `isqrt` integration inside the ID/EX stage.

Figure 1. Architectural view of the CVE2 processor and the `isqrt` custom extension. Rather than routing the instruction to an external `CV-X-IF` coprocessor, `isqrt` executes directly inside the ID/EX ALU datapath and reuses the standard write-back multiplexer.

Although CVE2 includes top-level ports for the CORE-V eXtension Interface (`CV-X-IF`, shown greyed out at the top of Figure 1a), the baseline core sets parameter `XInterface = 1'b0` and leaves handshake handling incomplete. Implementing `isqrt` directly inside the existing ALU avoids altering the top-level module port list, keeps `tb/` untouched, and modifies only three RTL files:

1. `cve2/rtl/cve2_pkg.sv`: adds `OPCODE_CUSTOM_0 = 7'h0b` (`opcode_e`) and `ALU_ISQRT` (`alu_op_e`).
2. `cve2/rtl/cve2_decoder.sv`: decodes `OPCODE_CUSTOM_0`, verifies `funct7 == 0`, `rs2 == 0`, and `funct3 == 0`, asserts `rf_ren_a_o = 1'b1` and `rf_we = 1'b1`, routes `rs1` via `OP_A_REG_A`, and selects `ALU_ISQRT` (or asserts `illegal_insn = 1'b1` on any mismatch).
3. `cve2/rtl/cve2_alu.sv`: computes the 16-step restoring root combinationally from `operand_a_i` and selects `isqrt_result` onto `result_o` when `operator_i == ALU_ISQRT`.

Cycle-exact write-back timing. Because CVE2 merges decode, operand read, ALU execution, and write-back setup into a single ID/EX stage, an `isqrt` instruction that enters ID/EX at rising clock edge t_k reads `rf_rdata_a` (`rs1`) asynchronously, propagates through the ALU combinational network during cycle k, and drives `rf_wdata_id_o = alu_result` into the write-back multiplexer (`cve2_wb.sv`) with `rf_we_wb_o = 1'b1` and `instr_done_o = 1'b1`. The flip-flop register file captures the new root on the next rising clock edge t_{k+1}. Because `rs1` is read continuously during cycle k and the destination flip-flop only updates on the clock edge t_{k+1}, in-place instructions (`isqrt x11, x11`) operate without read-after-write corruption, and writes to `x0` are ignored by the hardwired zero register in `cve2_register_file_ff.sv`.

2 Arithmetic derivation and RTL implementation

The hardware datapath implements a restoring, digit-recurrence (radix-4) square-root algorithm that consumes the 32-bit unsigned input x two bits at a time, from the most significant pair `x[31:30]` down to `x[1:0]`. While non-restoring variants eliminate the conditional restore multiplexer [3], the restoring formulation maps cleanly onto a single unsigned comparator and subtractor per stage in SystemVerilog.

Let $b_k \in \{0, 1, 2, 3\}$ denote the k-th 2-bit pair from the top of x, and let $P_k = \sum_{j=0}^{k-1} b_j 4^{k-1-j}$ be the $2k$-bit prefix shifted in after k steps ($P_0 = 0$, $P_{16} = x$). At the end of step k, the algorithm maintains a k-bit integer partial root Q_k and an unsigned remainder R_k satisfying the loop invariant:

$$P_k = Q_k^2 + R_k, \qquad 0 \le R_k \le 2Q_k, \qquad (Q_0 = 0,\ R_0 = 0). \tag{2}$$

In step $k+1$, shifting in the next 2-bit group b_k scales the prefix to $P_{k+1} = 4P_k + b_k = (2Q_k)^2 + (4R_k + b_k)$. Because $R_k \le 2Q_k$ and $b_k \le 3$, we have $4R_k + b_k \le 8Q_k + 3 < (2Q_k + 2)^2$ - $(2Q_k)^2$, which proves that the new integer root Q_{k+1} can only be $2Q_k$ or $2Q_k + 1$. Since $(2Q_k + 1)^2$ - $(2Q_k)^2 = 4Q_k + 1$, a single comparison between the shifted remainder $T = 4R_k + b_k$ and the trial divisor $D = 4Q_k + 1$ determines the next root digit:

$$T = 4R_k + b_k, \quad D = 4Q_k + 1, \qquad (Q_{k+1},\ R_{k+1}) = \begin{cases} (2Q_k + 1,\ T - D), & \text{if } T \ge D, \\ (2Q_k,\ T), & \text{if } T < D. \end{cases} \tag{3}$$

Both branches preserve (2): if $T < D$, then $R_{k+1} = T \le 4Q_k = 2Q_{k+1}$; if $T \ge D$, then $R_{k+1} = T - (4Q_k + 1) \le (8Q_k + 3) - (4Q_k + 1) = 4Q_k + 2 = 2(2Q_k + 1) = 2Q_{k+1}$. After $k = 16$ iterations,

$P_{16} = x$ and $Q_{16}^2 \le x = Q_{16}^2 + R_{16} \le Q_{16}^2 + 2Q_{16} < (Q_{16}+1)^2$, which guarantees $Q_{16} = \lfloor \sqrt{x} \rfloor$ for all 2^{32} unsigned inputs.

SystemVerilog implementation and bit-width bounds

The two listings below show the complete decoder and ALU additions in `cve2_decoder.sv` and `cve2_alu.sv`.

```
// cve2_decoder.sv
OPCODE_CUSTOM_0: begin
  if (instr[31:25] == 7'b000_0000 &&
      instr[24:20] == 5'b0_0000   &&
      instr[14:12] == 3'b000) begin
    rf_ren_a_o = 1'b1;   // read rs1
    rf_we      = 1'b1;   // write rd
  end else begin
    illegal_insn = 1'b1; // trap
  end
end
// ... ALU control part of the decoder
OPCODE_CUSTOM_0: begin
  alu_op_a_mux_sel_o = OP_A_REG_A;
  alu_op_b_mux_sel_o = OP_B_REG_B; // unused
  alu_operator_o     = ALU_ISQRT;
end
```

```
// cve2_alu.sv  (sq_* are 32-bit logic)
sq_x = operand_a_i;  sq_root = '0;  sq_rem = '0;
for (int i = 0; i < 16; i++) begin
  sq_rem   = (sq_rem << 2) | (sq_x >> 30);
  sq_x     = sq_x << 2;
  sq_trial = (sq_root << 2) | 32'h1;  // D
  sq_root  = sq_root << 1;
  if (sq_rem >= sq_trial) begin       // T >= D
    sq_rem  = sq_rem - sq_trial;
    sq_root = sq_root | 32'h1;
  end
end
isqrt_result = sq_root;
// result mux
ALU_ISQRT: result_o = isqrt_result;
```

Because the `for` loop has constant bounds (`0..15`) inside an `always_comb` block, synthesis unrolls it into sixteen cascaded combinational stages with no intermediate flip-flops. Using 32-bit unsigned vectors (`logic [31:0]`) for `sq_rem`, `sq_trial`, and `sq_root` is overflow-safe at every stage: entering the final iteration ($i = 15$), $Q_{15} \le 2^{15} - 1 = 32{,}767$ and $R_{15} \le 2Q_{15} = 65{,}534$, so the shifted remainder $T = 4R_{15} + b_{15} \le 262{,}139 < 2^{18}$ and the trial value $D = 4Q_{15} + 1 \le 131{,}069 < 2^{17}$ require at most 18 bits. Synthesis constant-propagates the unused upper bits away, while Figure 2 traces the unrolled 16-stage cascade on the concrete example $x = 20$.

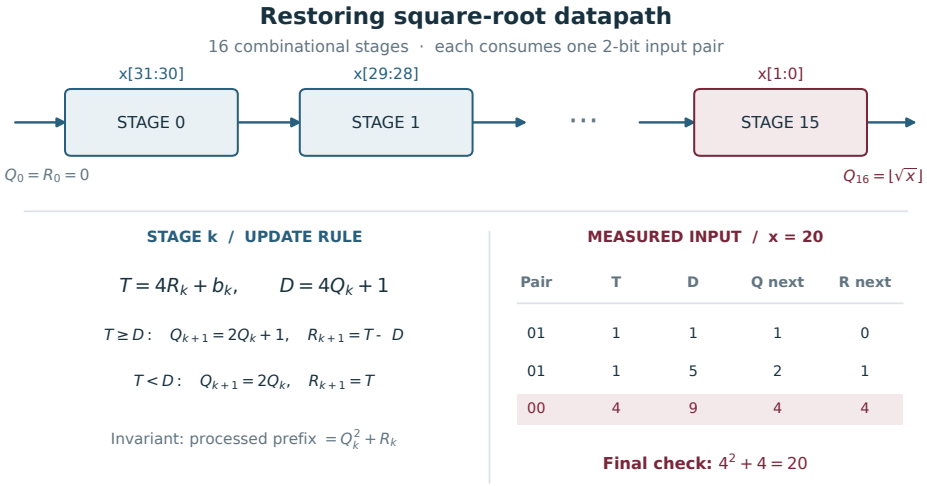


Figure 2. Unrolled 16-stage restoring radix-4 datapath and concrete trace for $x = 20$ (`0x00000014`). The first thirteen 2-bit pairs are zero ($Q_{13} = R_{13} = 0$); the last three pairs (`00`, `01`, `00`) yield $20 = 4^2 + 4$, driving `isqrt_result = 4` onto the ALU result bus in the same clock cycle.

3 Functional verification and performance evaluation

All verification artefacts were rebuilt from a clean export of repository revision `95090a4` using Verilator 5.054, the bare-metal RISC-V GNU toolchain (`riscv64-elf-gcc` 16.2.0, compiled for `-march=rv32imc -mabi=ilp32`), and GNU Make 3.81, without reusing prebuilt simulator binaries or ELF images. Table 2 summarises the four verification programs executed on the Verilator testbench (`build/sim/Vtb_top`) [6], [7].

Table 2. Verification suite summary. `sw_isqrt` and `hw_isqrt` are the supplied task benchmarks; `isqrt_corners` and `isqrt_integration` were developed to verify boundary accuracy, hazard forwarding, and illegal-opcode traps.

Program	Checks	Verification scope and pass criteria
`sw_isqrt`	5	Supplied software baseline; checks 5 known roots and runs the 256-input benchmark
`hw_isqrt`	2,007	Supplied hardware test; checks 16 directed cases, 1,000 random values against `sw_isqrt(x)`, 1,000 range-checked values, in-place `isqrt x11, x11`, and back-to-back dependent roots
`isqrt_corners`	43	New directed corner suite: $x \in \{0, 1, 2, 3, 4, 2^{31}-1, 2^{31}, 65535^2-1, 65535^2, 2^{32}-2, 2^{32}-1\}$, `rd = x0` discard, `rs1 = x0`, register-pressure patterns, and 4 malformed `custom-0` encodings
`isqrt_integration`	200,705	New exhaustive-boundary and integration suite: r^2-1, r^2, r^2+1 for every $r \in [1, 65{,}535]$ (196,607 checks), 4,096 pseudo-random 32-bit inputs, and mixed RV32IMC dependency chains

Independent 64-bit interval oracle and trap verification. Relying solely on the supplied software routine as a reference can mask shared overflow bugs near $2^{32}-1$. In `sw/isqrt_integration.c`, every hardware output $r = \texttt{hw_isqrt}(x)$ is checked directly against the mathematical definition (1) using 64-bit unsigned arithmetic (`uint64_t(r) * r <= x && uint64_t(r + 1) * (r + 1) > x`), preventing wrap-around when $r = 65{,}535$. To verify that `isqrt` interacts cleanly with neighboring instructions in the two-stage pipeline, `isqrt_integration.c` also runs an inline assembly block that chains `lw` → `isqrt` → `sw` → `lw` → `isqrt` → `mul` → `divu` → `c.addi` (16-bit compressed instruction) → `isqrt`. Finally, `sw/isqrt_corners.c` installs a custom machine trap vector (`mtvec`) and executes four malformed `custom-0` words (`funct7 = 1`, `funct3 = 1`, `rs2 = 1`, and a combined invalid pattern), verifying that each raises `mcause = 2` (illegal instruction), captures the exact faulting instruction in `mtval`, leaves the architectural destination register untouched, and resumes normal execution when the handler advances `mepc` by 4 bytes.

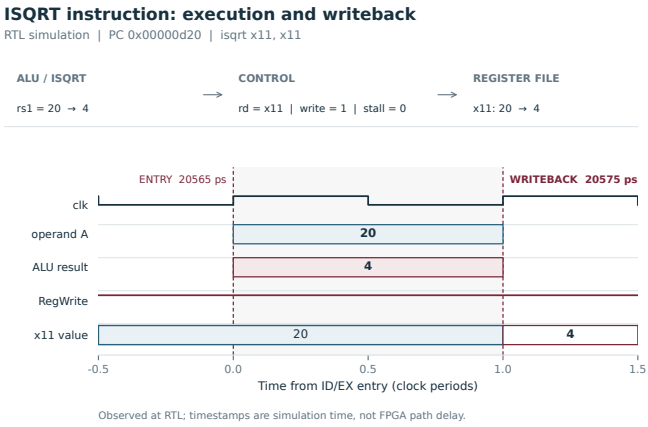


Figure 3. Cycle-accurate Verilator waveform (`build/sim/trace_core.fst`) for the in-place instruction `isqrt x11, x11` at PC `0x00000d20` during `hw_isqrt`. While the instruction is in ID/EX, `rf_rdata_a = 20` and `alu_result = 4` settle in the same cycle; at the next rising clock edge, `x11` updates from 20 to 4 with `rf_we_wb = 1`, `id_stall = 0`, and `illegal_insn = 0`.

Benchmark speed-up over software

To compare cycle counts fairly, both `sw_isqrt.c` and `hw_isqrt.c` seed the same deterministic PRNG (`rand_seed(7)`), evaluate 256 pseudo-random 32-bit inputs inside the timed benchmark loop, and accumulate the roots into a volatile checksum, producing the exact same value (`10996072`). Across the 256 roots, the software bit-by-bit loop requires **63,234 total cycles** (**247 cycles per root**), whereas the hardware instruction completes the benchmark in **5,913 total cycles** (**23 cycles per root**)—a **10.7**× end-to-end speed-up (90.7% cycle reduction). The remaining 22 cycles per iteration are spent in the C loop control, PRNG updates, and checksum accumulation; the `isqrt` instruction itself executes in a single ID/EX cycle (Figure 3).

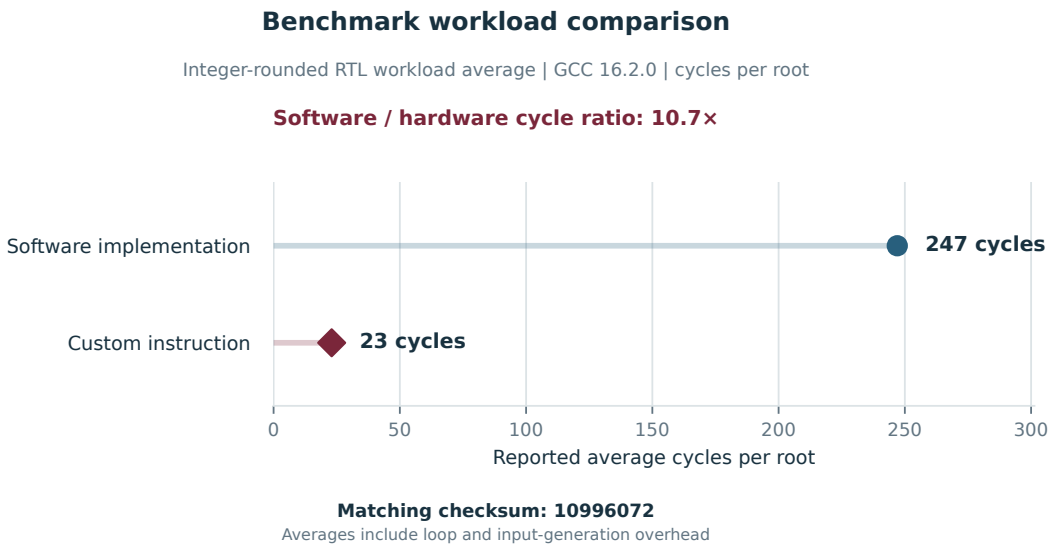


Figure 4. Average cycles per square root on the 256-sample workload (`sw_isqrt` vs. `hw_isqrt`).

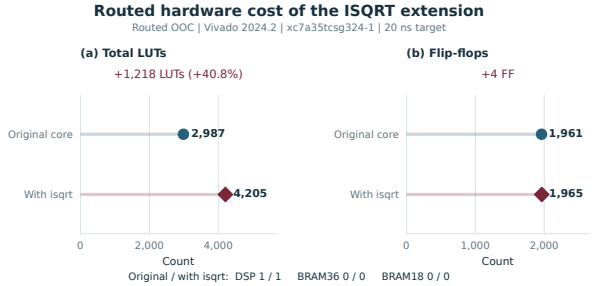
4 FPGA synthesis and place-and-route evaluation

To quantify the physical cost of the unrolled 16-stage datapath, both the unmodified baseline core and the `isqrt`-extended core were synthesised, placed, and routed with AMD/Xilinx Vivado 2024.2 targeting an Artix-7 `xc7a35tcsg324-1` under a **20 ns (50 MHz) primary clock constraint**. Because bare CVE2 exposes more top-level signals than the physical I/O pins of the CSG324 package, both variants are built in an identical out-of-context (OOC) wrapper flow (`fpga/cve2_fpga_top.sv`, `fpga/synth.tcl`, `fpga/implement.tcl`):

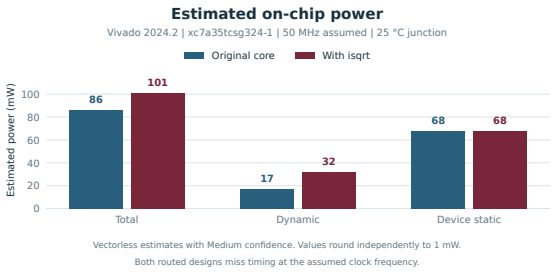
- **Core configuration:** RV32IMC (`RV32M = RV32MFast`, `RV32E = 0`), flip-flop register file (`RegFile = RegFileFF`), `MHPMCounterNum = 0`, and synchronised active-low reset.
- **Clock-gate replacement:** The behavioural simulation latch in `tb/cve2_clock_gate.sv` is replaced in `fpga/cve2_clock_gate.sv` by a Xilinx 7-series `BUFGCE` primitive, and `SYNTHESIS` is defined so simulation-only RVFI/tracer logic is excluded.
- **Constraints and implementation stages:** `fpga/core.xdc` defines a 20.0 ns clock on `clk_i` and uniform input/output delays (4.0 ns max, 0.0 ns min). Both runs execute `synth_design -mode out_of_context`, `opt_design`, `place_design`, `phys_opt_design`, and `route_design` with default Vivado directives. The baseline checkpoint is generated by restoring `cve2_pkg.sv`, `cve2_decoder.sv`, and `cve2_alu.sv` from the initial repository import commit `af1e7b4`.

Table 3. Post-synthesis and post-route FPGA comparison on Artix-7 `xc7a35tcsg324-1` (whole CVE2 core plus `cve2_fpga_top` wrapper, 20 ns target clock period).

Metric	Original core	With ISQRT	Change
LUTs (post-synthesis / post-route)	3,034 / 2,987	4,178 / 4,205	+1,218 routed (+40.8%)
LUT utilisation of `xc7a35t` (20,800 LUTs)	14.36%	20.22%	+5.86 percentage points
Flip-flops (post-route)	1,961	1,965	+4 (+0.2%)
DSP48E1 / RAMB36 / RAMB18 blocks	1 / 0 / 0	1 / 0 / 0	unchanged
Worst setup slack, WNS (ns, overall routed)	-0.884	-29.556	-28.672
Internal reg-to-reg setup slack (ns, routed)	+0.058	-29.556	-29.614
Worst hold slack, WHS (ns, overall routed)	-2.232	-2.168	+0.064 (similar)
Total estimated on-chip power at 50 MHz (mW)	86	101	+15 (+17.4%)
Dynamic / device static power (mW)	17 / 68	32 / 68	+15 / 0



(a) Routed LUTs and flip-flops.



(b) Estimated on-chip power at 50 MHz.

Figure 5. Physical impact of the unrolled `isqrt` datapath on the Artix-7. Nearly all additional area is combinational LUT logic (+1,218 routed LUTs) implementing the sixteen compare-and-subtract stages; sequential storage increases by only 4 flip-flops and DSP/BRAM counts remain unchanged.

Timing closure, stage progression, and power analysis

Both designs route all routable nets and report zero DRC errors, although DRC warnings remain. Neither achieves full timing closure at 50 MHz under the out-of-context setup. In the extended core, the worst reported internal register-to-register path starts at the LSU register `handle_misaligned_q_reg_replica` and ends at register-file bit `g_rf_flops[8].rf_reg_q_reg[8][0]`. It passes through the combinational `isqrt_comb` network. The saved timing report records a **49.387 ns** data path delay and **-29.556 ns** setup slack [6]. Slack includes clock skew, uncertainty, and the destination setup requirement; it is not simply the clock period minus data path delay.

In the baseline, the worst reported register-to-register path has +0.058 ns setup slack and a 19.672 ns data path delay. It runs from `instr_rdata_id_o_reg[6]` to the prefetch FIFO register `rdata_q_reg[0][10]` clock-enable pin. The input-to-output path with -0.884 ns slack runs from `irq_fast_i[7]` to `instr_addr_o[2]`, with 4 ns input and output delay constraints. Both runs also have hold violations (-2.232 ns and -2.168 ns). The external clock context is incomplete (`HD.CLK_SRC` is unset), and ports lack physical partition-pin locations. The router also reports `Constraints 18-8777`: required files for tile splitting are unavailable. These results do not establish an operating clock frequency or board timing closure [4], [6]. Figure 6 compares the recorded synthesis and routing results.

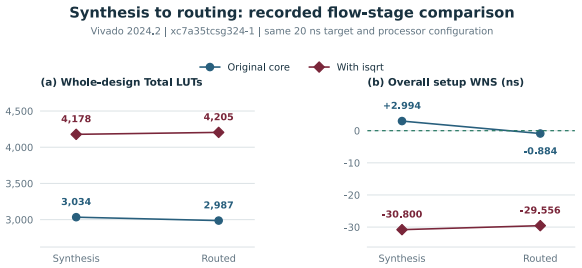
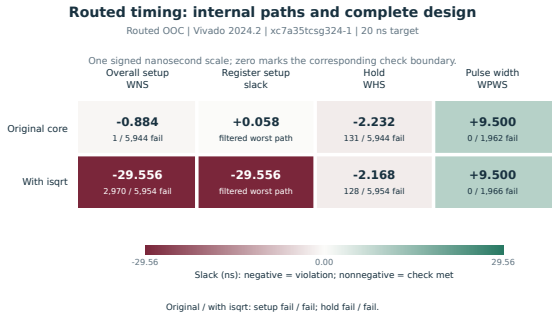


Figure 6. Synthesis-to-routing progression for LUTs and worst setup slack (WNS). Total LUT counts remain nearly constant through physical implementation, whereas the baseline WNS drops from +2.994 ns post-synthesis to -0.884 ns post-route once interconnect delays are included. For the extended core, the -30.800 ns post-synthesis WNS confirms that the 16-stage combinational chain breaks the 20 ns budget even before placement and routing.

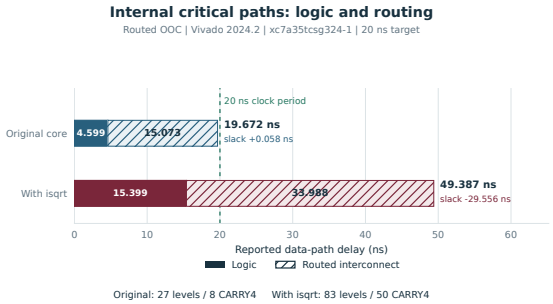
Power estimation. Power figures in Table 3 and Figure 5b are produced by Vivado `report_power` in vectorless mode on the routed checkpoints (typical process, 25 °C ambient, 50 MHz clock, 12.5% default primary-input toggle rate, Medium confidence level) [5]. Device static power is fixed at 68 mW for the `xc7a35t` package, while dynamic power rises from 17 mW to 32 mW (+15 mW) due to switching activity along the deep combinational carry chains.

5 Architectural trade-offs and an iterative multi-cycle alternative

The single-cycle unrolled architecture was chosen for the initial implementation because it preserves CVE2's single-cycle ALU execution model: it requires zero state registers, zero controller changes, and finishes every `isqrt` in one ID/EX cycle. However, Figure 7 exposes the fundamental physical limitation of full spatial unrolling on an FPGA fabric.



(a) Post-route setup and hold slack summary.



(b) Logic vs. routing delay on the worst path.

Figure 7. Post-route timing breakdown. In the extended core, the critical path spans 83 logic levels (including 50 `CARRY4` primitives and 33 LUTs), accumulating 15.399 ns of cell logic delay (31.2%) and 33.988 ns of interconnect routing delay (68.8%), compared with 27 logic levels (4.599 ns logic, 15.073 ns routing) in the baseline [4].

Because stage $k+1$ cannot decide whether $T \geq D$ until stage k has finished subtracting D from T and updating R_k, all sixteen carry chains are connected in series. On an Artix-7, routing between slices accounts for nearly 70% of the 49.387 ns path delay. Moreover, because the endpoints of the baseline and extended critical paths differ and include surrounding decode/LSU logic, the 49.387 ns figure represents the worst full core path through the ALU rather than the isolated delay of the square-root block alone.

Proposed alternative: 16-cycle iterative restoring unit

Instead of instantiating sixteen copies of the recurrence hardware in series, a multi-cycle iterative unit folds equation (3) in time across sixteen clock cycles using a single comparator/subtractor stage and four local registers: shifted radicand `sq_x_q` (32 bits), partial root `sq_root_q` (16 bits), running remainder `sq_rem_q` (18 bits), and a step counter `cnt_q` (4 bits). The proposal has not been simulated, synthesised, or routed. Table 4 gives unverified estimates for comparison.

Table 4. Implemented unrolled unit versus an unverified iterative proposal. All iterative figures are estimates; 50 MHz is a target.

Dimension	Unrolled combinational (implemented)	Iterative multi-cycle (proposed)
Execution latency	1 cycle in ID/EX	16–18 cycles estimated, including start/finish
Area overhead	+1,218 routed LUTs (+40.8%), +4 FFs	Rough estimate: 80–120 LUTs, 70–100 FFs; requires synthesis
Critical path	83 logic levels (50 `CARRY4`), 49.387 ns; fails 20 ns target	One 18-bit compare/subtract; 20 ns target remains unverified
Pipeline control	Purely combinational; no stall or FSM required	Requires `start`/`busy`/`valid` handshake; stalls ID/EX via `ex_valid_o` just like `cve2_multdiv_fast.sv`
Benchmark cycles	23 cycles/root (10.7× vs. software)	39–40 cycles/root if 17–18 instruction + 22 overhead cycles; about 6.2× at equal clock
Wall-clock speed	Operating frequency unverified; timing fails at 50 MHz	Projected 0.80 μs at an assumed 50 MHz and 40 cycles/root; not measured

Controller integration and corner-case rules for the iterative unit. Integrating a multi-cycle `isqrt` unit cleanly into CVE2 requires following the exact stall protocol already used by `cve2_multdiv_fast.sv` and `cve2_id_stage.sv`:

1. **Operand capture (`rd == rs1`):** The unit must latch `operand_a_i` into `sq_x_q` on the initial start cycle so that holding ID/EX stalled across 16 cycles cannot corrupt the input or overwrite `rs1` prematurely.
2. **Single write-back and `x0` protection:** While `busy` is asserted, `ex_valid_o` is held low, stalling ID/EX (`id_in_ready_o = 1'b0`) and suppressing register write enable until the final iteration asserts `isqrt_valid_o = 1'b1` for exactly one cycle. Writes to `rd = x0` remain ignored by the register file.
3. **Illegal encodings and pipeline flushes:** Decoder validation runs before asserting `isqrt_start`,

so malformed `custom-0` instructions trap immediately without starting the FSM. If an external interrupt, debug request, or synchronous exception flushes ID/EX (`instr_first_cycle_id` or controller flush) while `isqrt` is busy, the FSM must abort immediately and clear `busy`/`valid` so a stale square root is never written to `rd` after trap entry.

As an intermediate design point, unrolling two radix-4 stages per cycle (radix-16, 8 cycles) or four stages per cycle (4 cycles) allows the designer to tune the exact balance between cycle count and maximum clock frequency.

6 Reproducibility and conclusion

Source code, tests, Tcl scripts, and archived reports are available in [6], [7]. Clone the full history to include baseline revision `af1e7b4`.

Functional simulation (Verilator 5.054, RISC-V GCC 16.2.0, GNU Make 3.81)

macOS uses `RISCV=riscv64-elf-`; Linux defaults to `riscv64-unknown-elf-`.

```
make test RISCV=riscv64-elf-                        # Supplied benchmarks
make run TEST=isqrt_corners RISCV=riscv64-elf-      # 43 corner/trap checks
make run TEST=isqrt_integration RISCV=riscv64-elf-  # 200,705 checks
```

FPGA synthesis, implementation, and power reproduction (Vivado 2024.2)

```
source /path/to/Vivado/2024.2/settings64.sh
python3 scripts/prepare_fpga_baseline.py

for v in extended baseline; do
  mkdir -p build/fpga/$v build/fpga/implementation/$v
  vivado -mode batch -source fpga/synth.tcl \
    -log build/fpga/$v/vivado.log -journal build/fpga/$v/vivado.jou \
    -tclargs xc7a35tcsg324-1 "$v"
  vivado -mode batch -source fpga/implement.tcl \
    -log build/fpga/implementation/$v/vivado.log \
    -journal build/fpga/implementation/$v/vivado.jou -tclargs "$v"
done

python3 -B scripts/audit_fpga_context.py --out-dir output/fpga/context_audit/ubuntu
python3 -B scripts/run_fpga_power.py --out-dir build/fpga/power_reproduction
```

Conclusion

Adding `isqrt rd, rs1` as an unrolled combinational unit inside CVE2 demonstrates both the software power and the hardware cost of single-cycle custom instructions: it achieves a 10.7× benchmark speed-up with minimal RTL invasiveness (three files modified, zero pipeline control changes) and passes over 200,000 verification checks, but increases core LUT count by 40.8% and creates a 49.4 ns critical path on Artix-7. For clock-constrained FPGA deployments, a 16-cycle iterative unit is a candidate for reducing combinational depth. Under the stated cycle assumptions, its projected cycle advantage is about 6.2×; its area, timing closure, and wall-clock performance require implementation and measurement.

Note. AI tools were used only for language review and editorial suggestions when expanding this report; all architecture design, RTL implementation, verification, and FPGA analysis are the author's own work.

References

[1] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, version 20240411, Sec. 35.3.2, Apr. 2024. [Online]. Available: https://docs.riscv.org/reference/isa/v20240411/.

[2] OpenHW Foundation, *CORE-V CVE2 User Manual*, "Pipeline Details" and "Instruction Decode and Execute," draft documentation. [Online]. Available: https://docs.openhwgroup.org/projects/cve2-user-manual/en/latest/03_reference/pipeline_details.html.

[3] J. O'Leary, M. Leeser, J. Hickey, and M. Aagaard, "Non-restoring integer square root: A case study in design by principled optimization," in *Theorem Provers in Circuit Design*, LNCS vol. 901. Springer, 1995, pp. 52–71. doi: https://doi.org/10.1007/3-540-59047-1_42.

[4] AMD, *Vivado Design Suite User Guide: Design Analysis and Closure Techniques*, UG906, 2024.2 ed., Dec. 2024. [Online]. Available: https://docs.amd.com/r/2024.2-English/ug906-vivado-design-analysis.

[5] AMD, *Vivado Design Suite User Guide: Power Analysis and Optimization*, UG907, 2024.2 ed., Nov. 2024. [Online]. Available: https://docs.amd.com/r/2024.2-English/ug907-vivado-power-analysis-optimization.

[6] A. Ez-Zouine, *CVE2 Custom ISA Extension: Source Code and FPGA Evidence*, rev. b24909e, 2026: https://github.com/ahmedez-zouine/cve2-custom-isa-extension.

[7] A. Ez-Zouine, "ISQRT verification and benchmark artifacts," rev. 95090a4/b24909e, 2026: https://github.com/ahmedez-zouine/cve2-custom-isa-extension/tree/Master/output.